Evaluación de Unidad

1. Datos Generales

Asignatura	Álgebra Lineal
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	Edwin René Guamán Quinche
Nombre del Estudiante	Pineda Jiménez Yordy Javier
Fecha	04 al 10 de noviembre de 2026
Horario	-----

2. Objetivo:

- Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

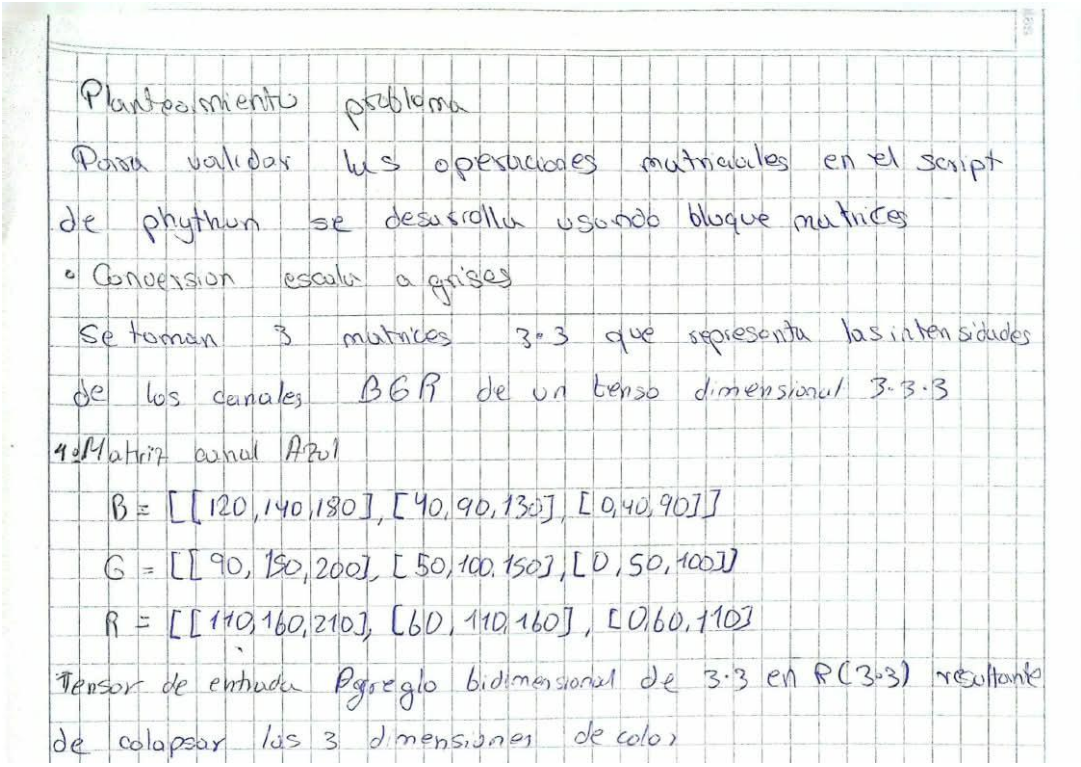
El tema elegido es **Procesamiento de Imágenes**

En la historia siempre ha sido muy importante preservar las experiencias humanas en forma de memoria. Pero la memoria no siempre cumple este rol. Por eso la imagen ha surgido como una herramienta importante para preservar estos eventos. En este caso el procesamiento de las imágenes, juegan un papel importante para mantener la imagen en una buena calidad y poder almacenarla de manera efectiva. Se hace usando métodos como

el filtrado de suavizado, realce de contraste, o escala a grises. Sistemas hechos por computadora ayudan a realizar estas operaciones de matrices complejas.

En términos de Ingeniería Informática el proyecto está relacionado con el procesamiento de imágenes donde se puede ver que las imágenes se representan como objetos matemáticos, es decir, una matriz bidimensional R^(m x n). La alteración de la luminosidad de las imágenes, el filtrado espacial son solo ejemplos particulares de la aplicación de la álgebra lineal como el producto de convolución, las transformaciones afines y las técnicas de descomposición de matrices. Gracias al enfoque práctico de la Ingeniería Informática y basado en los principios de los sistemas y las matrices en la computación [1] así como la algebraización de los algoritmos [2].

4. Desarrollo de las operaciones



2º Procedimiento paso a paso

Se aplica la combinacion podevada

A_Pbalt= 0.114 * B + 0,587 • G + 0,299 • R

Multiplicacion escalar de cada canal

0.114 • B = [[13.68, 13.96, 20.52], [4.56, 10,26, 14.82] [00.456,0.26]

0.587 • G = [[529, 88.05, 117.40], [29.35, 58.7, 88.05] [0.0, 29.35, 58.7]

0.299 • R = [[32.89, 47.84, 62.79], [17.94, 32.99, 47.84] [0.0, 17.9, 32.9]

Suma matricial

A_Pbat = [[99.40. 151.85, 200.71], [51.85, 101.85, 150.71] [0.00, 51.85, 101.85]

A= [[100, 150, 200], [50, 100, 150], [0, 50, 100]

Filtro suavizado

Se utiliza la matriz de agrises A y se le aplica un Kernel

Matriz de image A= 3*3 en R^(3*3)

Matriz Kernel K = cuadrada homogenea 3*3 (K = (1/9))

Procedimiento paso a paso

Se calcula el valor filtrado para cada central (1,1)

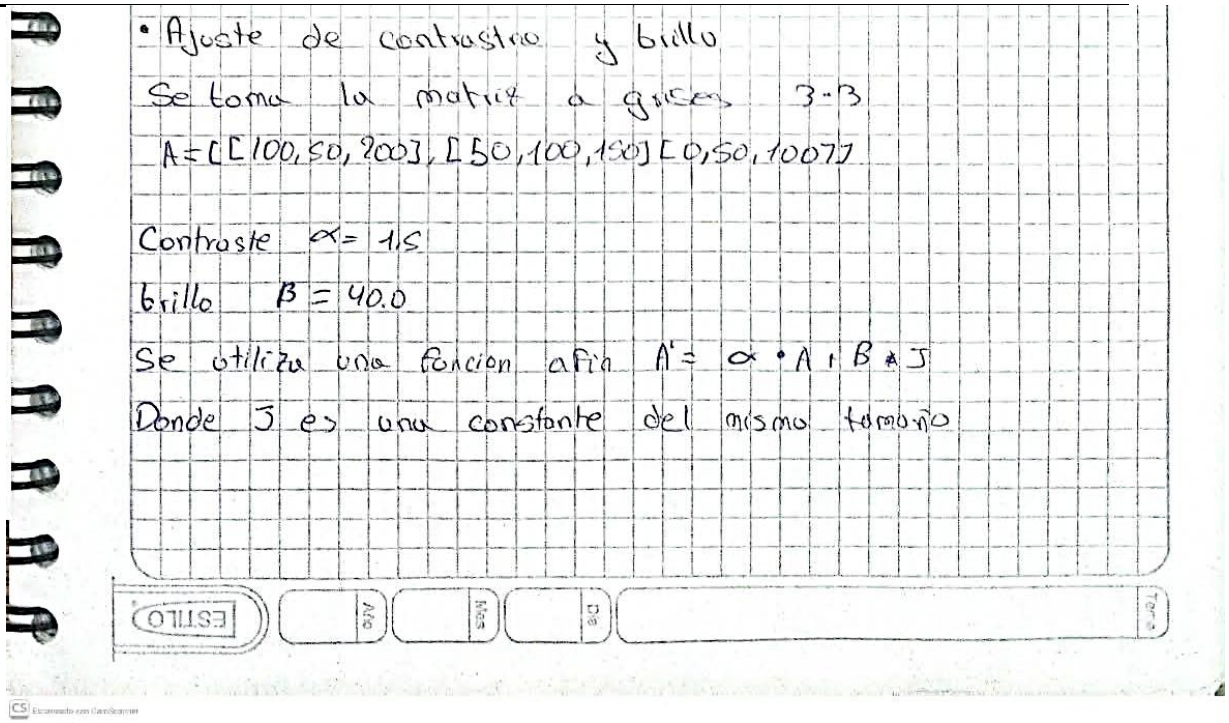
Producto Hadamard

M= A ado K = (1/9) • [[100, 150, 200], [50, 100, 150] [0, 50, 100]

valor suavizado = 11.11 + 16.67 + 22.22 + 5,56 + 11.11 + 16.67 10.0 + 5.56 + 11.11

valor suavizado = 100.02

Redonde = 100



- Ajuste de contraste y brillo

Se toma la matriz a grises 3-3

A=[[100,50,200],[50,100,150][0,50,100]]

Contraste $\alpha = 1.5$

brillo $\beta = 40.0$

Se utiliza una función afín $A' = \alpha \cdot A + B * J$

Donde J es una constante del mismo tamaño

Procedimiento paso a paso

Se multiplica el escalar α y se suma β

1.5 - A = 1.5 * [[100, 150, 200], [50, 100, 150], [0, 50, 100]]

(1.5 * A) + 40 = [[190, 265, 340], [115, 190, 265], [40, 115, 190]]

Recorte rango dinamico

Dado que las imagenes de 8 bits admite [0, 255]

Matriz Final = [[190, 255, 255][115, 190, 255], [40, 115, 190]]

5. Procedimiento / Metodología

Este proyecto se va a realizar con Python con Google Colab se fundamentan en transformar una matriz de píxeles para mejorar su calidad, extraer información o modificar su brillo entre otros

5.1 Descripción de la aplicación

La aplicación consiste en la manipulación digital de imágenes mediante la alteración de los valores numéricos contenidos en una matriz (m x n) a través de operaciones matemáticas del álgebra lineal [1], el análisis matricial permite transformar y extraer información de arreglos bidimensionales mediante operaciones bien definidas y formalizables algorítmicamente [2].

Este programa resuelve problemas:

1. Procesamiento de Imágenes: ¿Cómo se transforma una imagen?

- **Filtros de Suavizado (Paso Bajo):** Reducen el ruido promediando los píxeles vecinos mediante la convolución 2D con un kernel homogéneo de 5x5. El resultado es una imagen más uniforme, ideal para eliminar imperfecciones.
- **Transformaciones de Contraste y Brillo:** Modifican el rango dinámico y la luminosidad de la imagen aplicando una transformación afín sobre la matriz (multiplicación por escalar para el contraste y adición de constante para el brillo).

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	Python	3.11.x / 3.12.x
Entorno de desarrollo (IDE)	Google Colab	Entorno en la Nube
Librerías / Frameworks	NumPy matplotlib cv2 urllib	2.1.x 2.2.x 19.0.x
Plataforma de ejecución	Web	Navegadores Modernos

5.3 Funcionalidades implementadas

Funcionalidad 1: Conversión a Escala de Grises. Convierte el tensor tridimensional de color (BGR) en una matriz bidimensional m x n mediante la combinación lineal ponderada: Gris = 0.114B + 0.587G + 0.299*R.

Funcionalidad 2: Filtro de Suavizado por Convolución 2D. Aplica un núcleo (kernel) homogéneo de 5x5 con pesos 1/25 sobre la matriz de la imagen para atenuar variaciones bruscas de intensidad y reducir el ruido.

Funcionalidad 3: Transformación y Ajuste de Contraste y Brillo. Ajusta el brillo y el rango dinámico de la imagen mediante operaciones aritméticas sobre la matriz completa (alpha = 1.5 para contraste y beta = 40.0 para brillo), aplicando truncamiento del rango [0, 255].

5.4 Capturas de pantalla

Muestra la conversión de la imagen BGR a una matriz bidimensional m x n, analizando sus dimensiones y rango dinámico de píxeles.



Figura 1 Imagen Escala a Grises

Comparación visual entre la matriz de grises original y la matriz suavizada tras aplicar la convolución 2D con un kernel homogéneo de 5 x 5.



Figura 2 Filtro de Suavizado

Resultado de aplicar la transformación afín A' = α · A + β, modificando la matriz mediante adición y multiplicación por escalar.



Figura 3 Aplicación Contraste y Brillo

5.5 Código fuente (fragmentos clave)

Representación de datos multicanal (tensores) en matrices bidimensionales sobre el espacio R^(m x n).

```
# Convolución Automática
img_suavizada = cv2.filter2D(img_gray, -1, kernel_suavizado)

# ==========================================================
# CONVOLUCIÓN MANUAL (Filtro de Suavizado)
# ==========================================================
def convolucion_manual(imagen, kernel):
    m, n = imagen.shape
    k_h, k_w = kernel.shape
    pad_h = k_h // 2
    pad_w = k_w // 2

    imagen_padded = np.pad(imagen, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant', constant_values=0)
    salida = np.zeros_like(imagen, dtype=np.float32)

    for i in range(m):
        for j in range(n):
            region = imagen_padded[i:i+k_h, j:j+k_w]
            salida[i, j] = np.sum(region * kernel)
    return salida

img_suavizada_manual = convolucion_manual(img_gray, kernel_suavizado)
img_suavizada_manual = np.clip(img_suavizada_manual, 0, 255).astype(np.uint8)
```

Figura 1. Figura 1. 1 Conversión a escala a grises

Convolución discreta entre matrices y promediado ponderado de sub matrices adyacentes.

```python
# Convolución Automática
img_suavizada = cv2.filter2D(img_gray, -1, kernel_suavizado)

# ============================================================
# CONVOLUCIÓN MANUAL (Filtro de Suavizado)
# ============================================================
def convolucion_manual(imagen, kernel):
    m, n = imagen.shape
    k_h, k_w = kernel.shape
    pad_h = k_h // 2
    pad_w = k_w // 2

    imagen_padded = np.pad(imagen, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant', constant_values=0)
    salida = np.zeros_like(imagen, dtype=np.float32)

    for i in range(m):
        for j in range(n):
            region = imagen_padded[i:i+k_h, j:j+k_w]
            salida[i, j] = np.sum(region * kernel)
    return salida

img_suavizada_manual = convolucion_manual(img_gray, kernel_suavizado)
img_suavizada_manual = np.clip(img_suavizada_manual, 0, 255).astype(np.uint8)
```

Figura 1. 2 Filtro Suavizado

Transformaciones afines en espacios matriciales: multiplicación de una matriz por un escalar α y adición de matrices.

```python
# Transformación Automática usando operaciones vectorizadas
img_transformada = alpha * img_gray.astype(np.float32) + beta
img_transformada = np.clip(img_transformada, 0, 255).astype(np.uint8)

# ============================================================
# TRANSFORMACIÓN DE CONTRASTE Y BRILLO MANUAL (Píxel a Píxel)
# ============================================================
img_transformada_manual = np.zeros_like(img_gray, dtype=np.float32)
for i in range(img_gray.shape[0]):
    for j in range(img_gray.shape[1]):
        valor_nuevo = alpha * img_gray[i, j] + beta
        if valor_nuevo > 255:
            img_transformada_manual[i, j] = 255
        elif valor_nuevo < 0:
            img_transformada_manual[i, j] = 0
        else:
            img_transformada_manual[i, j] = valor_nuevo

img_transformada_manual = img_transformada_manual.astype(np.uint8)
```

Figura 1. 3 Brillo y contraste

Repositorio / Enlace al proyecto:

https://colab.research.google.com/drive/1wUh2_egW04DVMAx2EKykq2EQoC8RYw7L?usp=sharing

https://github.com/yordy2069/PrimerCicloU.git

6. Resultados

Reflexiona sobre los resultados obtenidos tanto en la parte manual como en la aplicación computacional. Responde las siguientes preguntas orientadoras:

- **¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?**

  Los resultados obtenidos mediante Python en Google Colab coinciden de forma exacta con el cálculo manual. La transformación afín arrojó el valor de 190 en la celda (1,1). La única diferencia radica en el uso de la función np.clip(), que realiza la evaluación del rango $[0, 255]$ de manera vectorizada.

- **¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?**

  En el cálculo manual, la mayor dificultad fue realizar el procedimiento ordenado de multiplicación escalar y suma de matrices constantes. En la programación, la mayor complejidad fue gestionar los tipos de datos (np.float32) para evitar el desbordamiento de memoria (overflow) antes de devolver la matriz a uint8.

- **¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?**

  Las transformaciones de contraste y filtros de suavizado son la base de los editores digitales de fotografía (como Photoshop), sistemas de mejora de imágenes médicas (rayos X) y preprocesamiento de cuadros en cámaras de seguridad.

- **¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?**

  Se consolidó la comprensión de que una imagen digital es una matriz de datos en un espacio vectorial m x n. Aprendí cómo conceptos teóricos abstractos del Álgebra Lineal (multiplicación por escalar, adición de matrices, productos celda a celda) se traducen directamente en código ejecutable para cambiar la apariencia de una imagen.

7. Conclusiones

- Expresar imágenes digitales como matrices de intensidad A dentro del espacio vectorial R^(m x

n) demuestra la utilidad inmediata del álgebra lineal en la ingeniería informática, donde diversas características visuales (como la luminosidad y el contraste) pueden ser modificadas usando operaciones algebraicas elementales como la multiplicación por un escalar y la suma de matrices.

- El filtrado por convolución discreta 2D mediante un kernel homogéneo de 5x5 demuestra cómo el promedio ponderado de submatrices atenúa las variaciones bruscas de intensidad, eliminando el ruido local de la imagen sin perder la estructura general del arreglo matricial.
- Se ha demostrado que las computaciones matriciales vectorizadas en NumPy son mucho más eficientes computacionalmente que el enfoque de iteración elemento a elemento, subrayando la importancia de optimizar los tipos de datos (uint8 versus float32) y los rangos dinámicos para prevenir los desbordamientos numéricos.

8. Referencias

- NumPy Developers, "NumPy Reference Guide: Linear Algebra (numpy.linalg)," Release 1.26.0, 2023. [En línea]. Disponible en: https://numpy.org/doc/stable/reference/routines.linalg.html
- W. Mora, C. Páez, M. Alfaro, E. Chacón, B. Rodríguez y R. Wynta, *Práctica del curso Álgebra Lineal para Computación: Selección de ejercicios*, Escuela de Matemática, Instituto Tecnológico de Costa Rica (ITCR), Revista digital Matemática, Educación e Internet, 2018. [En línea]. Disponible en: https://tecdigital.tec.ac.cr/revistamatematica/
- D. M. Russinoff, "A Formalization of Linear Algebra: Part I," *Formalized Mathematics / ACL2 Formalizations*, 2025. [En línea]. Disponible en: https://arxiv.org/abs/2507.19006